

TICKET BOOKING MANAGEMENT

School of Computing

**Bachelor of Technology
in
Computer Science & Engineering**

By

R.Balapoojitha (VTU24951)

10211CS224

Full Stack Application Development

Slot : E3-B09



**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
SCHOOL OF COMPUTING**

**VEL TECH RANGARAJAN Dr.SAGUNTHALA R&D
INSTITUTE OF SCIENCE AND TECHNOLOGY
(Deemed to be University Estd u/s 3 of UGC Act, 1956)
CHENNAI 600 062, TAMILNADU, INDIA**

May, 2026

BONAFIDE CERTIFICATE

It is certified that the work contained in the Full Stack Application Development report titled "Ticket Booking Management" by R.Balapoojitha (VTU24951) has been carried out in Winter semester of Academic year 2025-2026(WS2526).

Signature of Faculty

Dr. Hafizur Rahman

Assistant Professor

Computer Science & Engineering

School of Computing

Vel Tech Rangarajan Dr.Sagunthala R&D

Institute of Science and Technology

May, 2026

Signature of Course Coordinator

Dr.Sumithra.M

Assistant Professor(Senior Grade)

Computer Science & Engineering

School of Computing

Vel Tech Rangarajan Dr.Sagunthala R&D

Institute of Science and Technology

May, 2026

DECLARATION

I declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, we have adequately cited and referenced the original sources. I also declare that i have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission. I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

(R.Balapoojitha)

Date:06/05/2026

ABSTRACT

This project presents the design and development of a Ticket Booking Web Application built using the React.js framework. The application provides an interactive and user-friendly interface for users to browse available movies or events, select seats, and confirm bookings efficiently. The system is designed with multiple components such as home page, movie details, seat selection, and booking confirmation, ensuring smooth navigation using React Router.

Keywords:

Ticket Booking System

React.js

Web Application

Seat Selection

User Interface (UI)

User Experience (UX)

React Router

Frontend Development

Component-Based Architecture

Online Reservation System

LIST OF FIGURES

1.1	Image 1	4
3.1	Frontend Structure	7
3.2	Backend Structure	8
3.3	Image 2	9
4.1	Project architecture	12
4.2	Dataflow Diagram	13
4.3	Input Design	16

LIST OF TABLES

4.1	Example Comparison	15
7.1	Mapping of Project to Complex Engineering Problem (CEP)	20
7.2	Mapping of Project to Complex Engineering Activities (CEA)	21
7.3	SDG Mapping	21

LIST OF ACRONYMS AND ABBREVIATIONS

CI	Continuous Integration
ORM	Object Relational Mapping
API	Application Programming Interface
UI	User Interface
UX	User Experience
DB	Database

TABLE OF CONTENTS

	Page.No
ABSTRACT	iii
LIST OF FIGURES	iv
LIST OF TABLES	v
LIST OF ACRONYMS AND ABBREVIATIONS	vi
1 INTRODUCTION	1
1.1 Introduction	1
1.2 Aim of the Project	2
1.3 Scope of the Project	2
1.4 Framework	3
2 SURVEY OF SIMILAR APPLICATION IN MARKET	5
3 FRAMEWORK DESCRIPTION	6
3.1 Frontend Implementation	6

3.2	Backend Implementation	7
3.3	Database Implementation	9
4	METHODOLOGIES	11
4.1	Project Architecture	11
4.2	DataFlow Diagram	12
4.3	Module 1:User Interface	12
4.4	Module 2:Booking Management Module	13
4.5	Module 3:Database Management Module	13
5	RESULTS AND DISCUSSIONS	17
6	CONCLUSION AND FUTURE ENHANCEMENTS	18
6.1	Conclusion	18
6.2	Future Enhancements	18
	References	19
7	ADDRESSING COMPLEX ENGINEERING PROBLEM AND SDG	20
7.1	Mapping of the Project to Complex Engineering Prob- lem (CEP)	20
7.2	Mapping of the Project to Complex Engineering Ac- tivities (CEA)	21

7.3	SDG Mapping	21
-----	-----------------------	----

References	22
-------------------	-----------

Chapter 1

INTRODUCTION

1.1 Introduction

The Ticket Booking Web Application is developed to provide a simple and efficient way for users to book tickets online. Traditional booking methods are time-consuming and inconvenient, whereas this system allows users to easily browse available options, select seats, and confirm bookings from anywhere. This project is built using React.js to create a fast and interactive user [4]. It uses React Router for smooth navigation between pages without reloading. The application follows a component-based structure, making it easy to manage and extend. The main goal of this project is to develop a user-friendly, responsive, and attractive web application that simplifies the ticket booking process and improves the overall user experience.

1.2 Aim of the Project

The aim of this project is to develop a simple and user-friendly Ticket Booking Web Application that allows users to easily browse available events or movies, select seats, and confirm their bookings online. The application is built using React.js to provide a fast and interactive interface, along with React Router for smooth navigation between pages. The project focuses on creating an efficient, responsive, and visually appealing system that simplifies the ticket booking process and enhances the overall user experience.

1.3 Scope of the Project

The scope of this project is to develop a Ticket Booking Web Application that provides basic functionalities such as browsing available movies or events, viewing details, selecting seats, and confirming bookings. The application is built using React.js and uses React Router for smooth page navigation. It focuses on delivering a simple, responsive, and user-friendly interface. The project can be further extended by adding features such as user authentication, online payment integration, booking history, and backend database support for real-time data management.

1.4 Framework

Figure 1.1 shows the Ticket Booking system is developed using a modern web application framework that follows a client–server architecture. The frontend is built using React JS, which handles user interaction and displays the interface. The backend is developed using Node.js with Express.js, which processes user requests and manages business logic[2]. The database (MongoDB) is used to store user details, event information, and booking data[5]. The frontend communicates with the backend through APIs, and the backend interacts with the database to fetch and store data. This framework ensures smooth communication, fast performance, and easy scalability of the application.

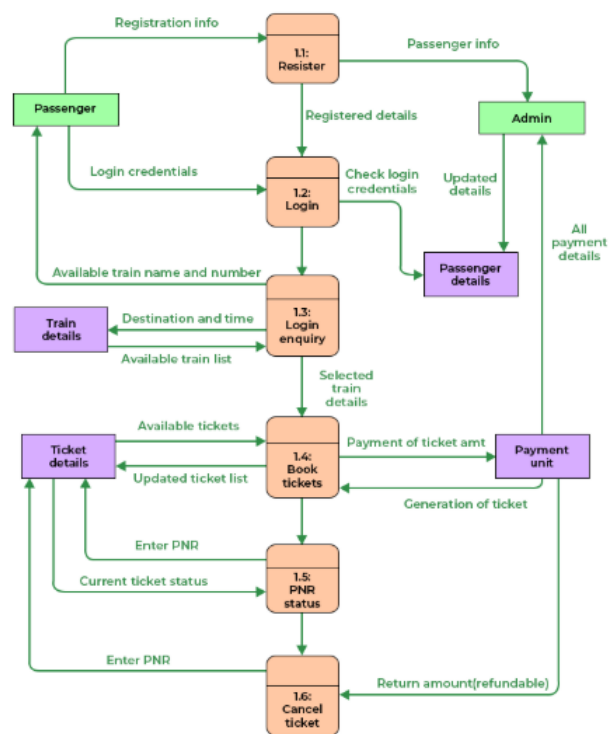


Figure 1.1: Image 1

Chapter 2

SURVEY OF SIMILAR APPLICATION IN MARKET

The ticket booking system has become an essential part of modern digital services, allowing users to conveniently book tickets for movies, travel, and events. Several popular applications such as Book-MyShow, redBus, IRCTC Rail Connect, MakeMyTrip, and Paytm provide efficient platforms for booking tickets online. These applications offer features such as event listings, seat selection, online payment, booking confirmation, and notifications. They help users save time and avoid long queues by enabling quick and easy reservations. Based on this survey, it is clear that a simple, user-friendly, and attractive ticket booking application can improve user experience by providing a clean interface, easy navigation[1].

Chapter 3

FRAMEWORK DESCRIPTION

3.1 Frontend Implementation

3.1.1 Environment Setup

The frontend of the Ticket Booking application was developed using React JS [4] to create an interactive and user-friendly interface. The environment was set up using Create React App, which provides a ready-made structure for development. Node.js and npm were used to install required libraries and run the application[2]. The project was coded using Visual Studio Code and tested in Google Chrome for smooth performance.

3.1.2 Structure of the Project

Figure 3.1 shows the project follows a simple and organized structure using React JS. All source code is placed inside the src folder, which contains subfolders like pages and components. The pages folder includes different screens such as Home, Booking, and Confir-

mation. The main file App.js manages navigation between pages using React Router. This structure makes the project easy to understand and maintain.

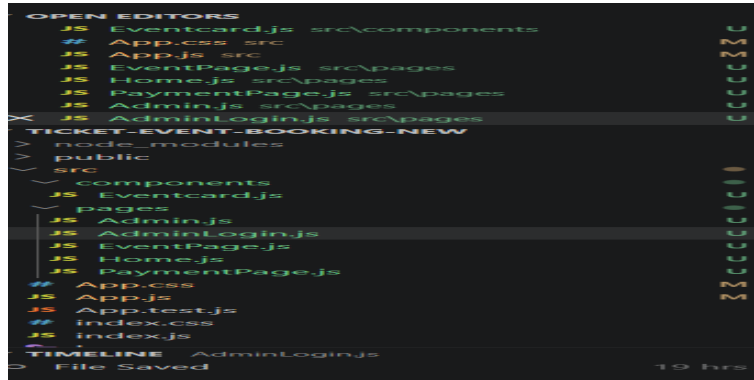


Figure 3.1: Frontend Structure

3.1.3 Execution Procedure

To run the application, the project is first opened in Visual Studio Code. Then, required dependencies are installed using npm. After that, the command `npm start` is executed in the terminal, which runs the project using Node.js. The application will automatically open in Google Chrome at `http://localhost:3000`, where users can interact with the ticket booking system.

3.2 Backend Implementation

3.2.1 Environment Setup

The backend of the Ticket Booking application was developed using Node.js[2] to handle server-side operations. The server was cre-

ated using Express.js, which simplifies building APIs and handling requests[2]. Required packages were installed using npm. The backend code was written and managed using Visual Studio Code and tested using tools like Postman to ensure proper functionality.

3.2.2 Structure of the Project

Figure 3.2 shows the backend project follows a clean and organized structure using Node.js. The main file, server.js, is used to start the server and define routes. Separate folders such as routes, models, and controllers are created to manage different functionalities. The routes folder handles API endpoints, controllers contain business logic, and models define data structure (if database is used). This modular structure helps in maintaining and scaling the application easily.



Figure 3.2: Backend Structure

3.2.3 Execution Procedure To run the backend server, the project is opened in Visual Studio Code. Then, dependencies are installed using npm. After that, the command `node server.js` or `npm start` is executed in the terminal to start the server using Node.js. Once the

server is running, APIs can be tested using Postman or integrated with the frontend for complete functionality.

```
C:\Users\rayal>cd ticket-event-booking-new

C:\Users\rayal\ticket-event-booking-new>npm start

> ticket-event-booking-new@0.1.0 start
> react-scripts start

(node:4088) [DEP0176] DeprecationWarning: fs.F_OK is deprecated, use fs.constants.F_OK instead
(Use 'node --trace-deprecation ...' to show where the warning was created)
(node:4088) [DEP_WEBPACK_DEV_SERVER_ON_AFTER_SETUP_MIDDLEWARE] DeprecationWarning: 'onAfterSetupMiddleware' option is deprecated. Please use the 'setupMiddlewares' option.
(node:4088) [DEP_WEBPACK_DEV_SERVER_ON_BEFORE_SETUP_MIDDLEWARE] DeprecationWarning: 'onBeforeSetupMiddleware' option is deprecated. Please use the 'setupMiddlewares' option.
Starting the development server...
Compiled successfully!
```

Figure 3.3: Image 2

3.3 Database Implementation

3.3.1 Database Configuration

Figure 3.1 shows about the Ticket Booking application uses a database to store user details, event information, and booking records. For this project, MongoDB is used as the database because it is flexible and easy to work with JSON data. The connection between the backend

and database is established using Mongoose in the Node.js environment. The database runs locally or on cloud platforms, and it ensures that all booking data is stored securely and can be retrieved when needed.

3.1.2 Schema Structure

In this project, the data is organized using schemas, which define the structure of the stored information. Using Mongoose, different schemas are created for users, events, and bookings. Each schema contains fields such as user name, email, event name, seat number, booking date, and payment status. This structured approach helps in maintaining consistency of data and makes it easier to manage and retrieve information from MongoDB.

3.1.3 Tables created

Instead of traditional tables, MongoDB uses collections to store data. In this project, three main collections are created: Users, Events, and Bookings. The Users collection stores user details like name and email. The Events collection stores event or movie information such as title, date, and ticket price. The Bookings collection stores booking details including selected seats, user ID, and event ID. These collections are linked logically to manage the complete ticket booking process efficiently.

Chapter 4

METHODOLOGIES

4.1 Project Architecture

Figure 4.1 shows the Ticket Booking application follows a simple and structured architecture using React JS for the frontend and Node.js with Express.js for the backend. The application is designed using a client–server model, where the frontend handles user interaction and the backend manages data processing and storage. The database MongoDB is used to store user and booking details. This architecture ensures smooth communication between different parts of the application and makes it easy to maintain and upgrade.

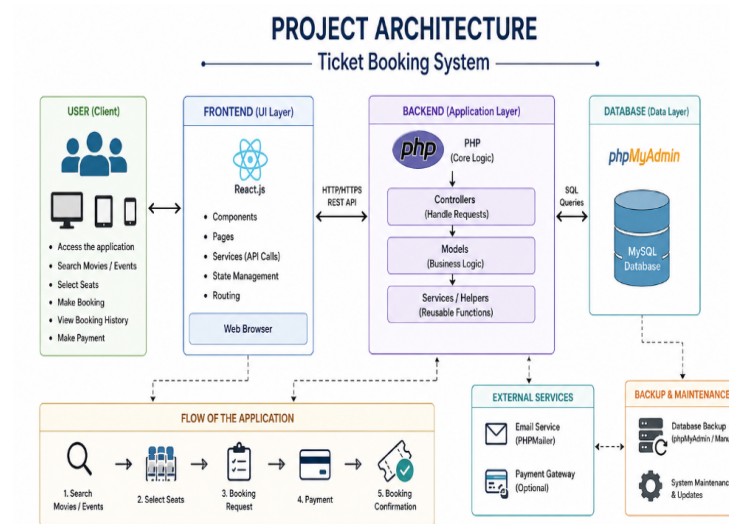


Figure 4.1: Project architecture

4.2 DataFlow Diagram

Figure 4.2 shows the data flow of the Ticket Booking system starts when the user opens the application and views available events. The user selects an event and chooses seats, then submits booking details. The frontend sends this data to the backend server built with Node.js. The backend processes the request and stores the booking information in MongoDB.

4.3 Module 1:User Interface

This module is responsible for designing and managing the user interface using React JS. It includes pages such as Home, Event Details, Booking, and Confirmation. Users can view available events, select seats, and navigate through the application easily. The module focuses

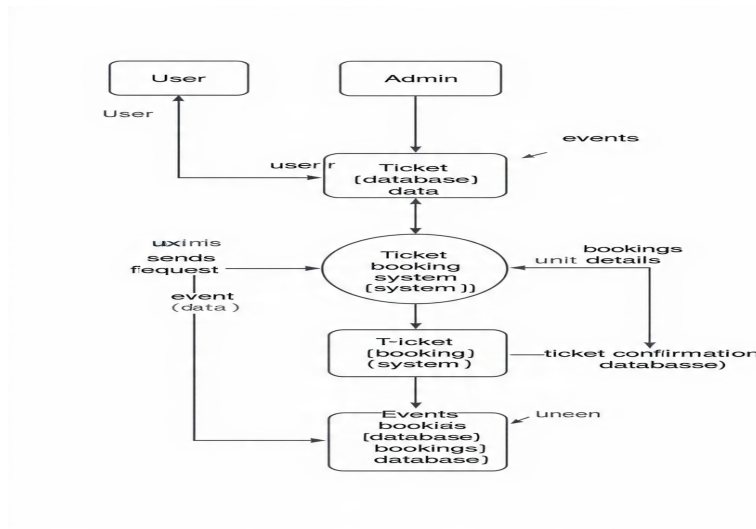


Figure 4.2: Dataflow Diagram

on providing an attractive design and user-friendly experience.

4.4 Module 2:Booking Management Module

This module handles the booking process using Node.js and Express.js. It receives user inputs from the frontend, processes booking requests, and validates seat availability. After processing, it stores the booking details in the database and sends a confirmation response to the user.

4.5 Module 3:Database Management Module

This module manages all data storage and retrieval operations using MongoDB. It stores information such as user details, event data, and booking records. Schemas are defined using Mongoose to maintain

structured data. This module ensures data consistency, security, and efficient access to information whenever required.

Advantages of this Project

a) Easy to Use Interface:

The application provides a simple and user-friendly interface developed using React JS, making it easy for users to navigate, select events, and book tickets without confusion.

b)Fast and Convenient Booking:

With the help of Node.js and Express.js, the system allows quick processing of booking requests, saving time and eliminating the need to stand in queues.

c) Secure and Organized Data Management:

The use of MongoDB ensures that all user and booking data is stored securely and managed efficiently, allowing easy retrieval and maintaining data consistency.

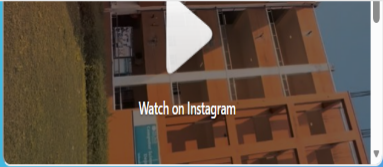
Disadvantages

- The application provides only basic ticket booking features.
- It requires a stable internet connection to work.
- Advanced security features are not fully implemented.

Content with screenshots

Table 4.1: Example Comparison

Applications	Advantages
BookMyShow	Easy movie and event booking with seat selection
redBus	Simple bus ticket booking with route details
IRCTC Rail Connect	Reliable train booking with seat availability
MakeMyTrip	Multiple services like flight, hotel, and travel booking
Ticket Booking System (Proposed)	Simple UI, fast booking process, and easy navigation



Watch on Instagram

Register

Name

Email

Department

Select Events:

- ☐ Debugging
- ☐ Quiz
- ☐ Connections
- ☐ Workshop
- ☐ Project Expo
- ☐ Expressive Art
- ☐ Other

Participants

[Proceed to Payment](#)

Figure 4.3: Input Design

Admin Panel

Event Name Price Image URL Video URL (.mp4) [Add Event](#)

Events



Tantraz

₹200

[Delete](#)



LIK pre-Release

₹150

[Delete](#)

Participants

Name	Email	Dept	Event	Participants	Total ₹
poojitha	rayala.balapoojitha@124gmail.com	CSE	Tantraz	2	₹200

Chapter 5

RESULTS AND DISCUSSIONS

The Ticket Booking web application was successfully developed and tested using modern technologies such as React JS, Node.js, and MongoDB. The system allows users to view available events, select seats, and complete bookings efficiently. The results show that the application provides a smooth and user-friendly interface, enabling users to navigate between pages such as Home, Booking, and Confirmation without difficulty. The seat selection feature works correctly, and booking details are stored and retrieved successfully from the database.

However, some limitations were observed, such as the absence of real-time seat updates and advanced security features. Overall, the application meets its objectives by providing a simple and effective ticket booking solution. Future improvements can enhance the system by adding more features like payment integration, authentication, and improved performance under high user load.

Chapter 6

CONCLUSION AND FUTURE ENHANCEMENTS

6.1 Conclusion

The Ticket Booking web application was successfully developed using modern technologies such as React JS for the frontend and Node.js with Express.js for the backend. The system provides a simple and user-friendly platform for users to view events, select seats, and confirm bookings easily. The project demonstrates how a basic ticket booking system can be designed with clean interface, smooth navigation, and efficient data handling, making the booking process quick and convenient.

6.2 Future Enhancements

The current system can be further improved by adding advanced features to enhance functionality and user experience. Future enhance-

ments may include integrating secure online payment gateways, implementing user authentication and login systems, and adding real-time seat availability updates. Features like email or SMS notifications, QR code-based ticket generation, and mobile responsiveness can also be included.

Chapter 7

ADDRESSING COMPLEX ENGINEERING PROBLEM AND SDG

7.1 Mapping of the Project to Complex Engineering Problem (CEP)

Level	Attribute	Characteristics	Wks	Justification
WP1	Depth of Knowledge	Requires in-depth engineering	WK4, WK5	Knowledge of React JS, hooks, validation, UI design.
WP2	Conflicting Requirements	Technical and non-technical conflicts	WK4, WK5	Balances usability with booking ticket constraints and validation.
WP3	Depth of Analysis	Analytical and design thinking	WK3, WK4	Component design, state handling, dynamic updates.
WP4	Familiarity	Partially new problems	WK4	Real-time booking and ticket updates.
WP5	Applicable Codes	Limited standard solutions	WK6	Custom booking and validation logic.
WP6	Stakeholder Needs	Multiple stakeholders involved	WK5, WK6	Students, faculty, organizers.
WP7	Interdependence	System-level integration	WK3, WK5	UI, state, and validation integration.

Table 7.1: Mapping of Project to Complex Engineering Problem (CEP)

7.2 Mapping of the Project to Complex Engineering Activities (CEA)

EA No.	Attribute	Characteristics	Justification based on the Project
EA1	Range of Resources	Use of technologies	React JS, HTML, CSS.
EA2	Level of Interactions	Complex module interaction	Event display, booking, and validation.
EA3	Innovation	Modern technologies	Hooks, conditional rendering.
EA4	Consequences	Impact on users	Simplifies booking process.
EA5	Familiarity	Beyond standard	Dynamic real-time system.

Table 7.2: Mapping of Project to Complex Engineering Activities (CEA)

7.3 SDG Mapping

SDG No.	SDG Title	Relevance to the Project
SDG 4	Quality Education	Supports academic events.
SDG 9	Innovation	Promotes React-based systems.
SDG 10	Reduced Inequalities	Accessible to all users.
SDG 11	Sustainable Communities	Encourages organized events.

Table 7.3: SDG Mapping

References

1. Eventbrite. (2026). Event Management and Ticketing Platform. Retrieved from <https://www.eventbrite.com>.
2. Google. (2026). Google Developers Documentation. Retrieved from <https://developers.google.com>.
3. PHPMailer. (2026). PHPMailer Documentation. Retrieved from <https://github.com/PHPMailer/PHPMailer>.
4. React. (2026). React Official Documentation. Retrieved from <https://react.dev>.
5. W3Schools. (2026). Web Development Tutorials. Retrieved from <https://www.w3schools.com>.
6. XAMPP. (2026). Apache Friends Documentation. Retrieved from <https://www.apachefriends.org>.